OutcomeServe: Crash-Consistent Recovery for Long-Horizon Agentic LLM Serving

Shirley Li [1] Lordphone Wen [1] Victoria Zhu [1]

1 Problem and Motivation

LLM agents alternate between model generation and tool calls. A worker crash can separate what an agent remembers from what its tools have already done: an order may be created just before the worker records the result. Retrying can create a duplicate order; restarting repeats completed inference and may repeat earlier side effects.

Research question. Can an agent-serving runtime resume after a worker crash while preserving committed tool effects and reducing repeated model computation? We propose OutcomeServe, a vLLM-based runtime that reconciles tool outcomes with a durable conversation log, then rebuilds inference state from the recovered token prefix. The challenges are resolving uncertain tool completion, aligning tool results with model context, and balancing durable logging overhead against recovery cost as context grows. When a non-idempotent tool outcome cannot be established, the runtime must halt and escalate rather than guess or blindly retry.

2 Related Work and Available Artifacts

AgentRewind restores aligned context and workspace checkpoints after unsuccessful trajectories (Zhuang et al., 2026); Crab selects filesystem checkpoint granularity for agent sandboxes (Wu et al., 2026). Their controlled-environment recovery does not resolve externally committed tool effects. Belayer coordinates rollout-worker and environment recovery for agentic RL, including rebuilding KV contents from logged token prefixes (Zhou et al., 2026). DéjàVu provides open-source KV-cache streaming and replication for inference recovery (Strati et al., 2024), without coordinating agent-level tool commits.

[1] 11-868 Large Language Model Systems. Correspondence to: Shirley Li <shirley3@andrew.cmu.edu>, Lordphone Wen <lordphow@andrew.cmu.edu>, Victoria Zhu <qiyuez@andrew.cmu.edu>.

11-868 Large Language Model Systems Project Proposal.

Durable workflow systems such as open-source LangGraph support checkpointed agent execution (LangChain, 2026). Recent work on agentic abstention studies when agents should stop acting once a task becomes infeasible (Luo et al., 2026); AgentAbstain further evaluates whether an agent withholds state-mutating commit tools before irreversible harm (Liu et al., 2026). OutcomeServe does not learn an abstention policy. Instead, it uses verifiable tool-commit evidence: recovery resumes only when a unique outcome can be established and otherwise enters a deterministic safe-halt state. Our contribution is a scoped integration and evaluation of tool-outcome reconciliation, recovery-time safe halt, and LLM re-prefill rather than a claim to invent durable execution.

We will use open-source vLLM (vLLM Project, 2026) and the Apache-2.0 Qwen3-4B model (Qwen Team, 2025); artifact links are included in the references.

3 Proposed Approach and Scope

Failure model. We target fail-stop crashes of the agent runner or inference process. The durable log and tool service survive, and only one runner owns a task at a time. Tools execute sequentially. For supported writes, the tool service atomically commits the side effect and its result under a stable operation ID and returns the stored result on duplicate submissions. We implement this contract using a transactional database. An external API without a query or deduplication contract cannot receive the same guarantee.

Commit and recovery protocol. The runtime combines an agent runner, vLLM, a durable log, a transactional tool service, and a fault injector:

1. Prepare. Before dispatch, persist the generated tool call, arguments, and operation ID derived from the task and step. Incomplete, uncommitted generation may be discarded after a crash.

2. Execute or reconcile. Submit the operation ID to the tool service. If the acknowledgment is lost,

resubmit the same ID: the service returns the committed result or atomically executes the operation once. A local PREPARED record alone does not prove non-execution.

3. Commit context or halt. If the service proves COMMITTED, append the stored result; if it proves NOT_COMMITTED, execute safely and append the result. If the outcome is UNKNOWN, persist RECOVERY_ABSTAIN, stop before any re-execution, and surface the operation ID and reason for manual resolution. A resumable prefix is submitted to vLLM for re-prefill before generation continues.

The invariant is that each completed step appears once in the committed transcript and matches one tool-side outcome; otherwise the task remains halted before further state-changing actions. The log stores token IDs and model versions. This preserves committed history without requiring future model outputs to be identical. GPU-cache replication, sandbox rollback, learned abstention policies, and storage failures are outside scope.

4 Evaluation Plan

Workloads and faults. We will build order tasks with 5, 10, and 20 tool steps, mixing reads, idempotent writes, and non-idempotent actions such as inventory decrements. Deterministic tool services and final-state verifiers make side effects auditable. Scripted traces isolate protocol correctness; real Qwen3-4B runs measure serving performance. We will vary prefixes over 2K, 8K, and 16K tokens and inject crashes during generation, before dispatch, after tool commit but before result logging, and after context commit. CPU tests will also crash the recovery path itself.

For recovery-time abstention, we construct paired cases with the same requested action: in the should-resume variant, the tool supports outcome query or operation-ID deduplication; in the should-abstain variant, a non-idempotent external action was dispatched but its outcome is unqueryable after acknowledgment loss. This pairing measures both unsafe retries and unnecessary halts without importing a separate abstention benchmark.

Comparisons. We compare full-task restart, naive retry, always halt, transcript checkpointing without tool reconciliation, and OutcomeServe's commit-aware resume-or-abstain protocol, plus failure-free runs with and without logging. External state persists across a crash. A separate resettable-environment restart experiment reports reset cost rather than treating rollback as free.

Metrics and hypotheses. Correctness measures are duplicate effects, tool-state mismatches, task success, correct-resume rate, false-abstention rate, unsafe-retry rate, and paired recovery accuracy. Performance measures are p95 crash-to-next-token latency, end-to-end task latency, re-prefilled and regenerated tokens, re-prefill throughput, GPU utilization, and failure-free overhead. Recovery time is split into worker startup, reconciliation, and re-prefill. We target zero observed duplicate effects for supported tools and expect safe halt to eliminate blind retries for unresolvable outcomes while avoiding the over-abstention of an always-halt policy.

Resume-versus-restart crossover. We ask when re-prefill-based resume stops outperforming full restart. We decompose

$$T_{\text{resume}} = T_{\text{startup}} + T_{\text{reconcile}} + T_{\text{prefill}}(L)$$

and

$$T_{\text{restart}} = T_{\text{startup}} + T_{\text{regenerate}}(k) + T_{\text{tools}}(k) + T_{\text{reset}},$$

where L is committed prefix length and k is completed tool steps. A heatmap of $\Delta T(L, k) = T_{\text{restart}} - T_{\text{resume}}$ will identify the crossover region.

5 Team and Milestones

Team. Shirley Li will coordinate the project and own the agent runner, vLLM integration, prefix replay, recovery controller, and UNKNOWN→RECOVERY_ABSTAIN transition. Lordphone Wen will own GPU/CPU infrastructure, worker supervision, deterministic fault and acknowledgment-loss injection, hardware monitoring, and performance benchmarks. Victoria Zhu will own the transactional tool service, durable-log schema, trace reconstruction, invariant checking, paired recovery workloads, and metric aggregation. All members share integration, experiments, and writing.

The minimum viable result compares full restart, naive retry, and crash-consistent resume for supported tools. Recovery-time abstention is a bounded extension that adds one terminal state, paired outcome-ambiguity cases, and associated correctness metrics without adding model training or a learned policy.

6 Infrastructure and Compute Budget

We request one L40S 48 GB or A100 40 GB GPU (or equivalent), 16–32 CPU cores, 64 GB RAM, and

Table 1. Planned eight-week schedule.

Week	Milestone
1	Finalize failure model and interfaces
2	Agent workload, paired cases, and verifier
3	vLLM integration and fault injection
4	Durable log and transactional tool service
5	Resume, safe-halt, and baseline implementations
6	Correctness experiments and debugging
7	Performance experiments and ablations
8	Analysis, demo, report, and presentation

200 GB NVMe storage. A second GPU enables a warm replacement replica; one GPU suffices for process-restart experiments. Qwen3-4B requires roughly 8 GB for BF16 weights before runtime overhead, with a 24 GB GPU as a fallback. We budget 90–120 GPU-hours: 20–30 for integration, 10–20 for pilots, 40–50 for evaluation, and 20 for repeated runs. CPU-only protocol and safe-halt tests reduce GPU use.

References

LangChain. LangGraph: Persistence, 2026. URL https://docs.langchain.com/oss/python/langgraph/persistence. Code: https://github.com/langchain-ai/langgraph. Accessed October 25, 2026.

Liu, X., Zhang, Y. E., Kasprova, V., Rabbani, P., Zahraei, P. S., Zhang, T., Ebrahimpour-Boroojeny, A., and Chandrasekaran, V. AgentAbstain: Do LLM agents know when not to act? In Advances in Neural Information Processing Systems, 2026. URL https://arxiv.org/abs/2607.10059. Evaluation and Datasets Track Oral. Code: https://github.com/AntiQuality/agentabstain.

Luo, H., Wen, B., and Wang, L. L. Agentic abstention: Do agents know when to stop instead of act?, 2026. URL https://arxiv.org/abs/2606.28733. NeurIPS 2026. Code: https://github.com/lhannnn/agentic-abstention.

Qwen Team. Qwen3-4B model card, 2025. URL https://huggingface.co/Qwen/Qwen3-4B. Open weights, Apache-2.0 license.

Strati, F., McAllister, S., Phanishayee, A., Tarnawski, J., and Klimovic, A. DéjàVu: KV-cache streaming for fast, fault-tolerant generative LLM serving, 2024. URL https://arxiv.org/abs/2403.01876. Code: https://github.com/msr-fiddle/dejavu.

vLLM Project. vLLM: A high-throughput and memory-efficient inference and serving engine, 2026. URL https://github.com/vllm-project/vllm. Open-source software.

Wu, T., Chang, C., Cao, L., Gao, W., and Wang, W. Crab: A semantics-aware checkpoint/restore runtime for agent sandboxes, 2026. URL https://arxiv.org/abs/2604.28138. Code: https://github.com/open-agent-infra/crab.

Zhou, J., Hu, Q., Sun, P., Zhang, X., and Zhang, W. Belayer: Efficient fault tolerance for LLM agentic RL training, 2026. URL https://arxiv.org/abs/2608.14635. Code: https://github.com/Zhoujiecheng/Belayer.

Zhuang, Y., Chen, K., Duan, Y., Zheng, S., Li, J., and Zhang, X.-Y. AgentRewind: Recoverable execution for long-horizon LLM agents, 2026. URL https://arxiv.org/abs/2608.14380. Code: https://github.com/Futuresis/replay-agent-recorder.